

Power of Ray Tracing, Calculus in your Hands: Developers, Lights, Vector, Action!

Submitted to:

Dr. Valentine Blez L. Lampayan
Mathematics Department
College of Science
Bulacan State University

in partial fulfillment of the requirements for the
course MAT201 - CALCULUS III of the program
BS Mathematics w/ specialization in
Computer Science.

by

Group Nine - BSM CS-2A
Chan Johrin Ashleigh SP.
Gulapa, Jose Miguel
Gutierrez, Kirby Benj D.
Palarca, Lywell B.
Tuazon, Allen Wilson D.

1st Semester, AY 2024-2025

Power of Ray Tracing, Calculus in your Hands: Developers, Lights, Vector, Action!

Group Nine - BSM CS-2A

Chan, Johrin Ashleigh Gulapa, Jose Miguel
Gutierrez, Kirby Benj D. Palarca, Lywell Tuazon, Allen Wilson D

Abstract

The power of Ray tracing technology completely transformed the advancement and beauty of computer graphics because it enabled the developers to produce incredibly quality and very lifelike images for animations and games. This paper studies and dive into the mathematical side of ray tracing, which are more focused on the functions of calculus and vector rendering procedures. The writers demonstrate the application of these mathematical ideas to the movement and simulations of lightning, shadows and reflections through real world type of applications and also theoretical ones. At its conclusion, this study shall highlights and emphasize the crucial relationship between mathematics and the pursuit to a better and more realistic graphics

1 Introduction

From pixels to full scale polygons, the gaming industry has evolved in an unprecedented way that allows just about anyone with access to the world wide web to bear witness to the wonders of technology today. With the evident desire to replicate reality and into the modes of entertainment by just about any game and media company, Ray Tracing (RTX) has become a household name amongst those that familiarize themselves with the development of graphic engines.

Many people who want to make games wonder: how much math do I need to know? The answer might surprise them: really not that much and what you do need is a lot of fun (Alfonso & Alfonso, 2023). Most that dream of being a developer whether it be

a game or another type of program get discouraged when they find out math that they are not that adept with is involved

The way this works is the light shoots and bounces around the environment and illuminates the area this is called global illumination, giving accurate reflections with no limitations, even causing actual optical magnification and illusions like rainbows naturally. (Quantic Animation, 2023)

Global illumination allows us to see objects not directly lighted.

Through this study we will be able to comprehend how graphics work. Mastering how to work with vectors in game engines is one of the core building blocks to becoming a great games programmer and gives you the ability to implement most of the mechanics found in modern games (Beaujean, 2023).

2 Role of Calculus and Vectors

Calculus and vectors play a significant role when it comes to rendering. This section explains how calculus and vectors are important in rendering.

2.1 Vectors in Rendering

Vectors are significant tools for representing and modifying objects in 3D space. A vector is a quantity which has both magnitude and direction. Those 2 quantities of vector makes it ideal for representing the position of the pixel on the screen and the direction of the light source which is very significant when it comes to rendering.

Vectors play an important role in rendering. It represents points and directions which is helpful in identifying points in 3D space that are relative to a specific origin and can also indicate the direction of a line or surface. Vectors also help with the lighting

calculation as it affects the light direction and lighting model which is very vital in rendering. Not only that, but vectors also affect the perspective that you see on screen. It defines the direction of where the angle of the camera is looking and the position of the object that is captured by the camera.

2.2 Calculus in Rendering

As stated in an article called MoldStud, An essential component for producing visually stunning gaming environments are realistic lighting and shadows. These effects can be really rendered and achieved with calculus. Calculus influences shadows and lighting in video games and animations by Ray tracing, Global illumination, Real-time lighting adjustment and many more.

Calculus contributes a lot to shape the visual effects of gaming and animations, lighting and shadows are just few of many. Calculus is responsible for the behavior and accuracy of the lighting. Through the use of it, the game developers and animators can create a realistic and visually pleasing world. Having accurate lighting and shadows, it will lead to a whole other level of watching and gaming experience.

3 Formula

Lighting, in general, is created with the single intention of adding value and creating a rich scene that enhances the mood or story of the scenario. Adding lighting and shadows make the image more 3-dimensional(3D) looks. But, the animators or game creators don't manually put the lighting and shadows, they are using mathematical formulas in order to do it.

This section discusses the formulas that are needed in order to calculate how the lights and shadows work in animations and games.

3.1 Vector Equation

Vectors carry a point A to point B. The direction of the vector AB is the path taken by the line that connects point A to B, and the length of the line that separates them is known as the vector's magnitude. Another name for vectors is a spatial vector or a Euclidean vector. In the domains of mathematics, physics, engineering, and many other subjects, vectors are used extensively.

In computer graphics, particularly in rendering, mathematical formulas are fundamental for simulating realistic lighting, shading, and reflections. Vectors and their operations, such as scalar products, will help calculate angles and directions of light source towards the object or surface (Pharr, Jakob, & Humphreys, 2016).

$$a \cdot b = |a| \times |b| \times \cos(\theta)$$

$$a \cdot b = a_x \times b_x + a_y \times b_y$$

If;

The angle is 0° : The intensity of light on the surface is maximum.

The angle is 90° : The intensity of light on the surface is minimal.

The angle is $< 90^\circ$:The intensity of light on the surface is zero.

3.2 Rendering Equation

According to Alfonso(2023) from a website called Quantic, “The term “rendering” refers to the process of creating a final image from a 3D model in animation or modeling software. The idea behind rendering is to simulate how a 3D object would appear in the real world”. The process of rendering is very crucial in the field of animation and game

development nowadays because the more realistic your graphics are, the game experience or the animated movie will also be better.

In order to create a realistic image in animations or games, the animators or the game creators use software that has the physics of light program in it. The one that stands out the most in this kind of field is the rendering equation.

$$L_o(x, \omega_o) = L_e(x, \omega_o) + f_r(x, \omega_i \rightarrow \omega_o) L_i(x, \omega_i) (\omega_i \cdot n)$$

Wherein;

$L_o(x, \omega_o)$ is an outgoing radiance at point x in outgoing direction ω_o

$L_e(x, \omega_o)$ is the emitted radiance at point x in outgoing direction ω_o

$f_r(x, \omega_i \rightarrow \omega_o)$ is the scattering function at point x from incoming direction ω_i to outgoing direction ω_o

$L_i(x, \omega_i)$ is the incoming radiance to point x from direction ω_i

$(\omega_i \cdot n)$ is the angle between the incoming direction and normal

4 Application

This section focuses on giving some examples and real life application of the rendering equation. This will give us some idea of how to utilize the rendering equation in a given situation and also helps us understand the importance of this given equation in the visuals of gaming and animation.

4.1 Graphics Rendering

Rendering is the heart of creating lifelike images in games and animations, translating raw 3D models into visually compelling scenes. This section explores the application of rendering techniques, focusing on the integration of mathematical formulas to replicate realistic lighting and shading. By examining these processes, we

gain a deeper appreciation of how developers achieve the stunning visuals that define modern digital media.

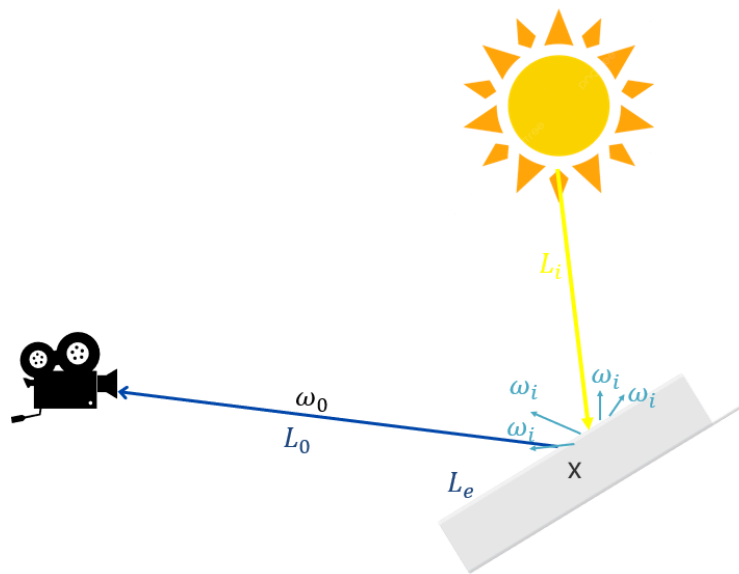


Figure 1
Visualization of the Rendering Equation

4.2 Calculation

Example calculation using the rendering equation:

Given:

Surface normal: $n = (0, 0, 1)$ (pointing upward).

Incoming light direction: $\omega_i = (0.6, 0.8, 0.2)$ (normalized direction from light).

Outgoing direction (view): $\omega_o = (0, 0, 1)$ (directly toward the viewer).

Light intensity (L_i): 1.0 (setted by the user in rendering app)

Diffuse BRDF (f_r): $\frac{Kd}{\pi}$ (Lambertian reflection), $Kd = 0.8$.

Emitted radiance (L_e): 0.

Solution :

1. Normalize ω_i

$$|\omega_i| = \sqrt{0.6^2 + 0.8^2 + 0.2^2} = \sqrt{1.04} \approx 1.02$$

$$\omega_i \text{ normalized} = \left(\frac{x}{|\omega_o|}, \frac{y}{|\omega_o|}, \frac{z}{|\omega_o|} \right)$$

$$\omega_i \text{ normalized} = \left(\frac{0.6}{1.02}, \frac{0.8}{1.02}, \frac{0.2}{1.02} \right)$$

$$\omega_i \text{ normalized} = (0.588, 0.784, 0.196)$$

2. Compute the dot product ($\omega_i \cdot n$)

$$\omega_i \cdot n = (0.588 \cdot 0) + (0.784 \cdot 0) + (0.196 \cdot 1) = 0.196$$

3. Evaluate (f_r)

For a Lambertian surface, the BRDF is:

$$f_r = \frac{K_d}{\pi} = \frac{0.8}{\pi} \approx 0.2546$$

4. Calculate the outgoing radiance L_o

$$L_o(x, \omega_o) = L_e(x, \omega_o) + f_r(x, \omega_i \rightarrow \omega_o) L_i(x, \omega_i) (\omega_i \cdot n)$$

$$L_o = 0 + 0.2546 \cdot 1 \cdot 0.196$$

$$L_o = 0.0499$$

The outgoing radiance is approximately 0.0499.

Realistic Problem: Lightning surface in a game

You are creating a scene for a video game in which a red marble tile on the floor is illuminated by a spotlight. There is no self-emitted light, and the tile reflects light diffusely (Lambertian reflection).

The light source and the camera (viewer) are positioned as follows:

Light source direction (ω_i): (1, 2, 2) *The light shines toward the tile from the direction.*

Viewer direction (ω_o): (0, 0, -1) *The viewer is looking straight down at the tile from the direction .*

Surface normal (n): $n = (0, 0, 1)$ *The tile's surface is flat and faces upward*

Other parameters:

Diffuse reflectivity (K_d): 0.9 *The tile reflects the incoming light diffusely.*

Incoming light intensity (L_i): 2.0 (arbitrary units) *The spotlight's intensity.*

Using the equation $L_o(x, \omega_o) = L_e(x, \omega_o) + f_r(x, \omega_i \rightarrow \omega_o) L_i(x, \omega_i) (\omega_i \cdot n)$

Calculate the outgoing radiance $L_o(x, \omega_o)$ seen by the player.

1. Normalize ω_i

$$|\omega_i| = \sqrt{1^2 + 2^2 + 2^2} = \sqrt{9} = 3$$

$$\omega_i \text{ normalized} = \left(\frac{x}{|\omega_i|}, \frac{y}{|\omega_i|}, \frac{z}{|\omega_i|} \right)$$

$$\omega_i \text{ normalized} = \left(\frac{1}{3}, \frac{2}{3}, \frac{2}{3} \right)$$

$$\omega_i \text{ normalized} = (0.333, 0.667, 0.667)$$

2. Compute the dot product ($\omega_i \cdot n$)

$$n = (0, 0, 1)$$

$$\omega_i \cdot n = (0.333 \cdot 0) + (0.667 \cdot 0) + (0.667 \cdot 1) = 0.667$$

$$\omega_i \cdot n = 0.667$$

3. Evaluate (f_r)

For a Lambertian surface, the BRDF is:

$$f_r = \frac{K_d}{\pi} = \frac{0.9}{\pi} \approx 0.286$$

4. Calculate the outgoing radiance L_o

$$L_o(x, \omega_o) = L_e(x, \omega_o) + f_r(x, \omega_i \rightarrow \omega_o) L_i(x, \omega_i) (\omega_i \cdot n)$$

$$L_o = 0 + 0.286 \cdot 2 \cdot 0.667$$

$$L_o = 0.286 \cdot 1.334 \approx 0.381$$

The player receives an outgoing radiance $L_o(x, \omega_o)$ of roughly **0.381 brightness units**. Taking into consideration the material's characteristics, the angle of incoming light, and lambertian reflection, this indicates that the marble tile reflects **38.1%** of the light intensity.

5. Summary and Conclusions

In summary, gaming and animation undergo a lot of changes in terms of visuals. By knowing the mathematical concepts like vectors and calculus and

then applying those concepts in the actual games and/or animations, the developers and animators will create a visual that is more realistic.

Rendering equations, like those used in this study, mathematically describe the interaction of light and surfaces. These equations enable developers to rely on physical principles expressed in software, instead of manually putting every light and shadow to the image.

All aspects of rendering, especially the lighting, shadows and reflection are getting upgraded thanks to the application of mathematical concepts and also the advancement of the technology. Combining these two significant things and we have a very stunning visual and the best gaming experience. This paper emphasizes that with the help of mathematical concepts like vectors and calculus, it is possible to achieve a visually stunning animation and game graphics.

References

Pharr, M., & Humphreys, G. (2004). *Physically based rendering: From theory to implementation*. <http://ci.nii.ac.jp/ncid/BB03369267>

Beaujean, M. (2023, December 9). *Vector Maths for game dev beginners*. <https://www.gamedeveloper.com/programming/vector-maths-for-game-dev-beginners>.

The role of Mathematics in Game Development | MoldStud. (2024, February 2). <https://moldstud.com/articles/p-the-role-of-mathematics-in-game-development>.

Alfonso, & Alfonso. (2023, March 16). What is rendering? - Quantic Animation. *Quantic Animation - The best outsourcing animation studio*. <https://quanticanimation.com/what-is-rendering>.

Beaujean, M. (2023, December 9). *Vector Maths for game dev beginners*. <https://www.gamedeveloper.com/programming/vector-maths-for-game-dev-beginners>.